VITYARTHI PROJECT REPORT

AI Fake Article Detector

Course: CSA 2001 – Fundamentals in AI/ML
Name: Mayank Anand
Reg No: 25BAI11209

---

1. What I Tried to Build

So for this project, I basically wanted to build something that can tell whether a piece of text is real or fake (AI-generated/spam). With tools like ChatGPT, Claude, etc., it's become super easy to generate convincing text, and honestly, sometimes it's hard to tell what's real anymore.

The goal here wasn't to make a perfect detector, but something practical that works reasonably well on normal articles and paragraphs.

---

2. Why This Problem Matters

Fake or AI-generated content isn't just a small issue anymore. It can:

- Spread misinformation really fast
- Influence people's opinions without them realizing
- Be used in spam or fake news campaigns

So even a basic tool that gives a probability of "real vs fake" can be useful. It's not about being 100% correct, but about giving a signal.

---

3. How I Approached It

I split the project into two parts:

1. Training the model
2. Making a simple web app to use it

Dataset

I used a dataset with two CSV files:

- One for real news
- One for fake news

I combined them, shuffled everything, and cleaned the text.

Text Cleaning

This part took more effort than expected. I removed:

- Punctuation
- URLs
- Numbers

At first I used pandas.apply() but it was acting weird, so I switched to a manual loop which worked better.

Vectorization

I used **TF-IDF** to convert text into numbers.
Basically, it turns words into weighted features so the model can understand patterns.

Model

I used **Logistic Regression**.

Nothing fancy, but:

- Fast
- Works well with TF-IDF
- Good enough accuracy

---

4. Web App (UI Part)

Instead of making a full frontend + backend setup, I used **Streamlit**.

It made everything way easier because:

- I could write UI and ML logic in one file
- No need for HTML/CSS/JS separately

Features I added:

- Text input box
- “Check Article” button
- Probability bar (real vs fake)

- Threshold slider (to adjust strictness)
- Confusion matrix graph

---

5. Key Decisions (And Why)

- **Logistic Regression instead of deep learning**
  → Faster, simpler, and honestly worked well enough
- **Limit TF-IDF to 5000 features**
  → Reduces memory usage and avoids overfitting
- **Use Streamlit**
  → Saved a lot of time compared to Flask/Django
- **Use caching (`@st.cache_resource`)**
  → So the model doesn't retrain every time

---

6. Problems I Faced

1. Data Cleaning Issues

Text was messy and inconsistent.
`apply()` wasn't reliable, so I had to manually clean it.

---

2. Memory Usage

Dataset was around 100+ MB.
TF-IDF transformation caused RAM spikes.

Fix:

- Dropped NaN values early
- Limited features

---

3. Weird Predictions on Small Text

Short inputs (like 2–3 words) gave random results.

Fix:

- Added a rule: minimum 5 words required

---

7. What the Model Does

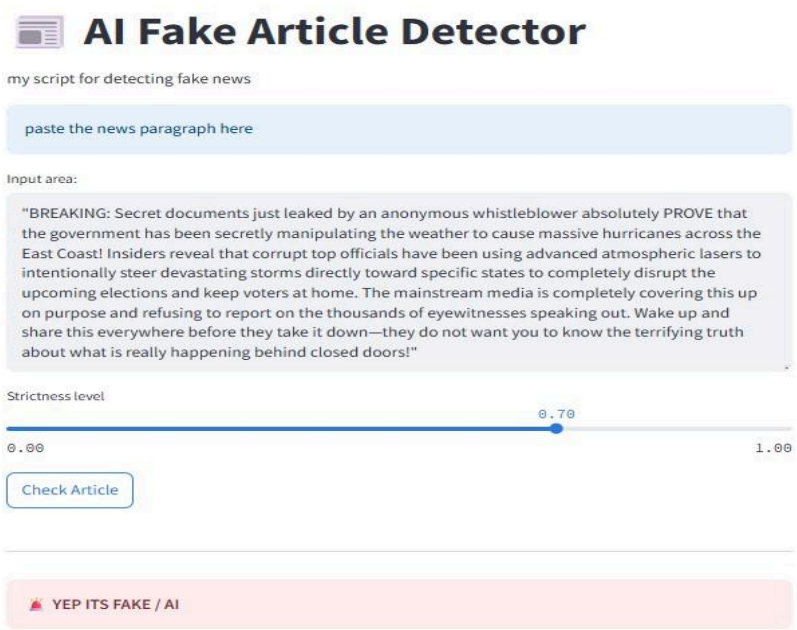
- Takes input text
- Processes it using the same TF-IDF pipeline
- Predicts:
    - Real OR Fake
- Shows probability score

It's not perfect, but works well for normal paragraphs.

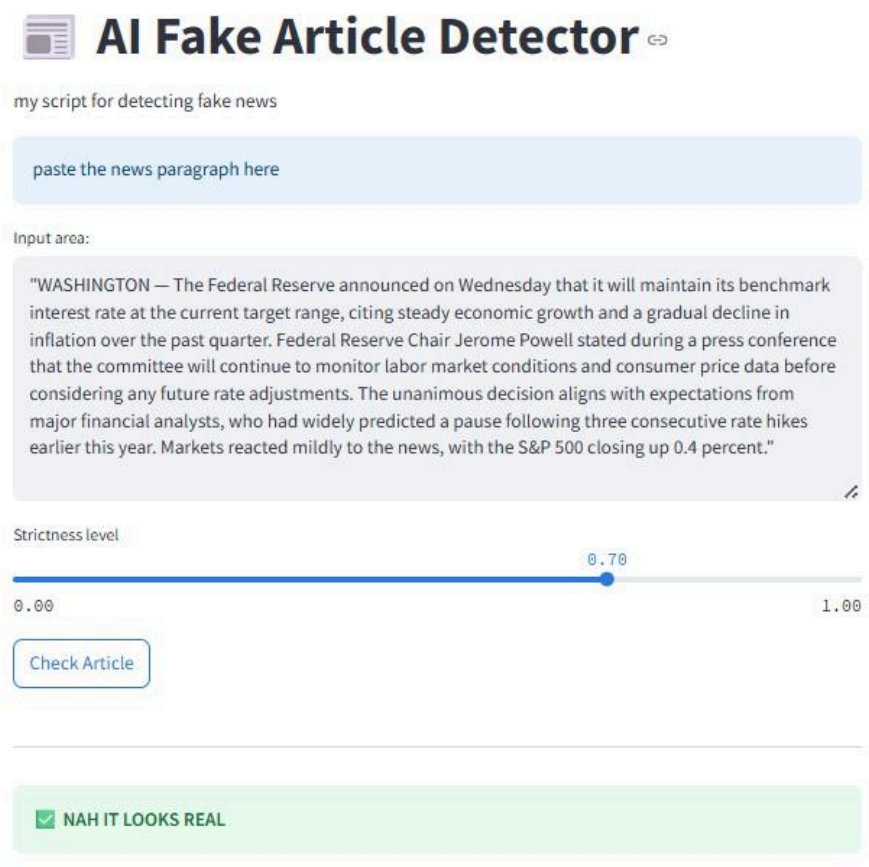
---

8. Screenshots

Fake Article Detection



Real Article Detection



---

9. What I Learned

This project actually helped me understand things better than just theory:

- How raw text is converted into numbers (TF-IDF)
- Why accuracy alone isn't enough (precision/recall matter)
- How to connect a model to a real UI
- How users actually interact with ML systems

---

10. Limitations

- Dataset is a bit outdated
- Struggles with very short text
- Model is basic (not deep learning)